Week 10: Capstone: plan and build

Plan before you code

- **Users:** name at least two kinds.
- **User story:** *As a ___, I want to ___ so that ___.*
- **MVP:** the smallest version that works and meets the brief. **Stretch:** everything else.
- **Database:** entities → ER diagram → tables in 3NF, with keys and constraints.
- **Route map:** page, URL, methods, SQL.

Page	URL	Method	SQL
List	`/`	GET	`SELECT ... JOIN`
Add	`/new`	GET, POST	`INSERT`
Edit	`/<id>/edit`	GET, POST	`UPDATE`
Delete	`/<id>/delete`	POST	`DELETE`
Report	`/report`	GET	`GROUP BY`

Do not store what you can calculate (availability, totals, age). Store the facts it is calculated from.

Start from the template

```
cp -r week-10-plan-and-build/capstone-template \
      my-work/capstone
cd my-work/capstone
flask --app app init-db     # builds instance/app.db
flask --app app run --debug
python3 -m unittest discover -s tests -v
```

`init-db` runs `schema.sql` then `seed.sql` and **deletes the old data**. Re-run it whenever you change either file.

File	Purpose
`PLAN.md`	Your plan
`schema.sql`	`CREATE TABLE`: parents first
`seed.sql`	Realistic sample data (8+ rows)
`app.py`	Routes and helpers
`templates/ static/`	Pages and CSS
`tests/`	Automated tests

Build a vertical slice

One feature through every layer: **table → SQL → route → template → test**. Prove each step before the next: try the SQL in `sqlite3` first.

For each form: the same four steps

validate → `with db:` → flash → redirect

Git: save and share

```
git status
git add my-work/capstone
git commit -m "Add kit list page"
#   say WHAT; start with a verb
git push

git switch -c feature/overdue       # new branch
git switch main
git merge feature/overdue
git log --oneline
```

- Commit small and often, after something works. At least **10 commits** are needed.
- Never commit secrets or the database file (`instance/` is ignored).
- Merge conflict: edit the file, delete the `<<<<<<< ======= >>>>>>>` lines, `git add`, `git commit`.

Capstone must-haves

- 2+ related tables in 3NF with constraints and a foreign key.
- CRUD, a search or filter, and a `GROUP BY`/join report.
- Server-side validation; `?` placeholders; POST for changes; 404 page.
- Base template, CSS, `url_for`, flash messages; realistic seed data.
- Week 11: 8+ tests, a README, 10+ commits.